Segmentación (pipeline) del procesador

1. Concepto de segmentación

La segmentación (*pipelining*) es una técnica de implementación que permite a la CPU trabajar sobre varias instrucciones a la vez, solapando sus etapas, en lugar de terminar una instrucción por completo antes de empezar la siguiente. No cambia el repertorio de instrucciones: el programa es el mismo; lo que cambia es la forma en que el hardware lo ejecuta.

Sin segmentación, la CPU recorre de forma secuencial todas las etapas de una instrucción (etapa 1, etapa 2, …, etapa k) y sólo entonces empieza con la siguiente.

Con segmentación, las etapas se organizan como una cadena de montaje y cada una trabaja sobre una instrucción distinta: mientras la instrucción 1 está en la etapa 3, la instrucción 2 está en la etapa 2 y la instrucción 3 en la etapa 1. En cada ciclo de reloj todas las instrucciones en curso avanzan una etapa a la vez.

1.1. Latencia y productividad

- **Latencia**: tiempo desde que una instrucción entra en el pipeline hasta que sale. La segmentación **no la reduce**; de hecho, suele aumentarla un poco por el retardo de los registros de segmentación y porque todas las etapas duran lo mismo que la más lenta.
- **Productividad** (*throughput*): número de instrucciones completadas por unidad de tiempo. Es lo que mejora la segmentación: en el caso ideal se completa **una instrucción por ciclo** (CPI = 1).

En la práctica no siempre se consigue CPI = 1, porque aparecen conflictos entre instrucciones, los **riesgos** o *hazards* (apartado 6), que obligan a introducir ciclos de espera o a aplicar técnicas de resolución.

2. Por qué RISC-V se adapta bien a la segmentación

RISC-V se diseñó pensando en implementaciones segmentadas eficientes. Varias decisiones del repertorio simplifican el pipeline:

- **Instrucciones de longitud fija (32 bits)** en RV32I: la etapa IF siempre lee 4 bytes y el siguiente PC secuencial es simplemente PC + 4. (La extensión comprimida «C» añade instrucciones de 16 bits; en este documento no se considera.)
- **Pocos formatos, con los campos en posiciones fijas**: rs1, rs2 y rd ocupan siempre los mismos bits, de modo que los registros fuente pueden leerse en ID a la vez que se decodifica la instrucción.
- **Arquitectura de carga/almacenamiento**: sólo las cargas y los almacenamientos (`lw`, `sw` y sus variantes) acceden a memoria. La dirección se calcula en EX y el acceso se hace en MEM; ninguna instrucción opera con la ALU sobre un dato de memoria.
- **Registro x0 cableado a cero**: escribir en x0 no tiene efecto. Esto simplifica la lógica de anticipación (nunca se anticipa x0) y permite codificar `nop` como `addi x0, x0, 0`.
- **Sin ranuras de retardo** (*delay slots*) en los saltos, a diferencia de MIPS (ver 6.3).

Formatos de instrucción de RV32I (posición de los campos en bits):

Formato	31–25	24–20	19–15	14–12	11–7	6–0	Ejemplos
R	funct7	rs2	rs1	funct3	rd	opcode	`add`, `sub`, `and`, `sll`
I	imm[11:0]		rs1	funct3	rd	opcode	`addi`, `lw`, `jalr`
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	`sw`, `sb`
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode	`beq`, `bne`, `blt`
U	imm[31:12]				rd	opcode	`lui`, `auipc`
J	imm[20|10:1|11|19:12]				rd	opcode	`jal`

3. Las cinco etapas del pipeline clásico

El modelo clásico (el de Patterson y Hennessy para RISC-V) divide la ejecución de cada instrucción en cinco etapas, cada una de un ciclo de reloj.

3.1. IF – Instruction Fetch (búsqueda de la instrucción)

- La CPU usa el PC (*Program Counter*) para leer de memoria la instrucción que toca ejecutar; normalmente se accede a la caché de instrucciones.
- En la misma etapa se calcula PC + 4 y se actualiza el PC, salvo que un salto indique otra dirección.
- La instrucción leída y su PC se guardan en el registro de segmentación IF/ID.

3.2. ID – Instruction Decode / Register Fetch (decodificación y lectura de registros)

- Se interpretan el opcode (bits 6–0) y los campos funct3/funct7 para saber qué operación es (suma, carga, salto…), y la unidad de control genera las señales de control.
- Se leen los registros fuente rs1 y rs2 del banco de registros.
- Se genera el inmediato, con extensión de signo, según el formato (I, S, B, U o J).
- La unidad de detección de riesgos comprueba si la instrucción depende de una carga anterior cuyo dato aún no está disponible (riesgo de carga-uso, apartado 6.2) y, en ese caso, detiene el pipeline.

3.3. EX – Execute / Address Calculation (ejecución)

- La ALU realiza la operación indicada: suma, resta, operación lógica, desplazamiento, comparación…
- En las instrucciones aritmético-lógicas aquí se calcula **el resultado definitivo** de la instrucción, aunque todavía no se haya escrito en el registro destino.
- En cargas y almacenamientos se calcula la **dirección efectiva**: rs1 + inmediato.
- En los saltos condicionales se comparan rs1 y rs2 y se calcula la dirección destino (PC + inmediato). En el diseño de referencia de este documento el salto se **resuelve** al final de esta etapa.
- En `jal` y `jalr` se calcula la dirección destino y el valor de retorno PC + 4.

3.4. MEM – Memory Access (acceso a memoria)

- En una carga (`lw`) se lee el dato de la memoria de datos (caché de datos).
- En un almacenamiento (`sw`) se escribe el valor de rs2 en la dirección calculada en EX.
- Las instrucciones aritmético-lógicas y los saltos no hacen nada en esta etapa: sólo pasan sus datos al registro MEM/WB.

3.5. WB – Write Back (escritura del resultado)

- Se escribe el resultado (el de la ALU o el dato leído de memoria) en el registro destino rd.
- `sw` y los saltos condicionales no escriben ningún registro. Una escritura en x0 se descarta.
- Tras esta etapa, el efecto de la instrucción ya es visible para el programador.

3.6. Qué hace cada tipo de instrucción en cada etapa

Instrucción	IF	ID	EX	MEM	WB
`add x5, x6, x7`	Leer instrucción; PC ← PC+4	Leer x6 y x7	x6 + x7	—	x5 ← resultado
`addi x5, x6, 12`	Ídem	Leer x6; inmediato 12	x6 + 12	—	x5 ← resultado
`lw x5, 8(x6)`	Ídem	Leer x6; inmediato 8	dir = x6 + 8	dato ← Mem[dir]	x5 ← dato
`sw x5, 8(x6)`	Ídem	Leer x5 y x6; inmediato 8	dir = x6 + 8	Mem[dir] ← x5	—
`beq x5, x6, L`	Ídem	Leer x5 y x6; inmediato	¿x5 = x6?; destino = PC + inm.	—	—
`jal x1, f`	Ídem	Inmediato	destino = PC + inm.; PC + 4	—	x1 ← PC + 4

3.7. Registros de segmentación

Entre cada par de etapas hay un **registro de segmentación** (IF/ID, ID/EX, EX/MEM y MEM/WB). Al final de cada ciclo, cada etapa deja ahí todo lo que la siguiente necesita: la instrucción, los operandos leídos, el resultado de la ALU, el número del registro destino y las señales de control. Sin estos registros, una etapa sobrescribiría los datos que la siguiente todavía está usando.

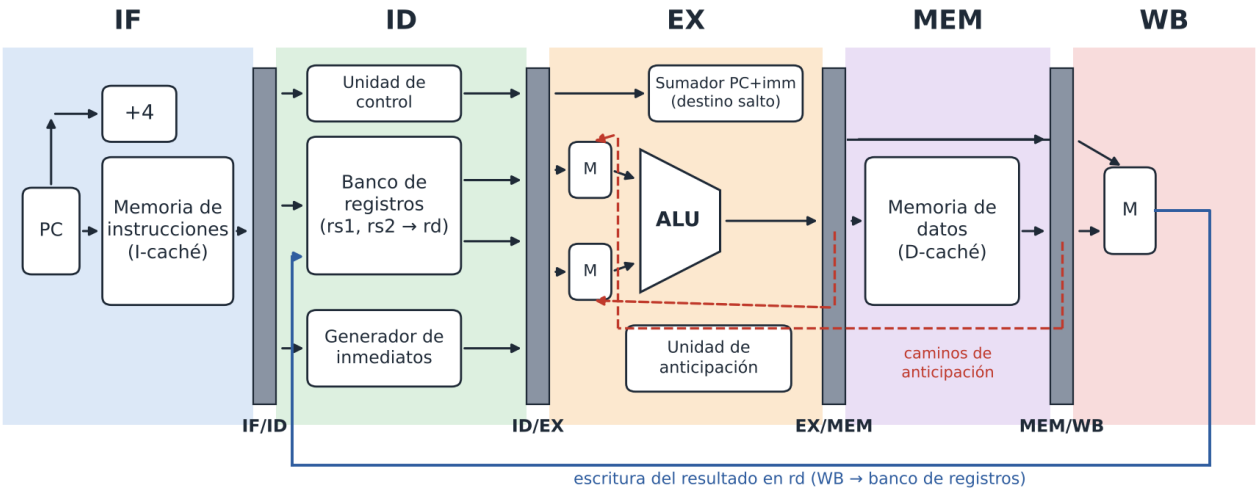


Figura 1. Ruta de datos segmentada simplificada de RISC-V, con los registros de segmentación y los caminos de anticipación (en rojo).

Dos detalles de diseño evitan conflictos internos:

- El banco de registros **escribe en la primera mitad del ciclo y lee en la segunda**. Así, si en un mismo ciclo una instrucción está en WB y otra en ID, la de ID ya lee el valor nuevo.
- Las señales de control se generan una sola vez, en ID, y viajan con la instrucción por los registros de segmentación; cada etapa usa sólo las suyas.

4. Funcionamiento temporal del pipeline

En un pipeline ideal de 5 etapas:

- Ciclo 1: la instrucción 1 está en IF.
- Ciclo 2: la instrucción 1 pasa a ID y la instrucción 2 entra en IF.
- Ciclo 3: la instrucción 1 está en EX, la 2 en ID y la 3 en IF.
- Ciclo 5: el pipeline está lleno —hay cinco instrucciones en curso, una por etapa— y la instrucción 1 termina (WB). A partir de aquí termina una instrucción en cada ciclo.

La forma habitual de representarlo es el **diagrama multiciclo**: cada fila es una instrucción y cada columna un ciclo de reloj. Ejemplo con cinco instrucciones RISC-V independientes:

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9
`add x5, x6, x7`	IF	ID	EX	MEM	WB				
`lw x8, 0(x10)`		IF	ID	EX	MEM	WB			
`sub x11, x12, x13`			IF	ID	EX	MEM	WB		
`sw x14, 8(x10)`				IF	ID	EX	MEM	WB	
`or x15, x16, x17`					IF	ID	EX	MEM	WB

Diagrama 1. Cinco instrucciones independientes: 5 + (5 − 1) = 9 ciclos.

Se distinguen tres fases:

- **Llenado** (ciclos 1–4): entran instrucciones, pero todavía no termina ninguna.
- **Régimen estacionario**: con el pipeline lleno, en cada ciclo termina una instrucción.
- **Vaciado**: ya no entran instrucciones nuevas y las que quedan van saliendo.

Cada instrucción sigue necesitando 5 ciclos (su latencia no cambia), pero, una vez lleno el pipeline, se completa una instrucción por ciclo.

5. Rendimiento del pipeline

5.1. Tiempo de ciclo

Todas las etapas avanzan al mismo ritmo, así que el reloj lo marca la etapa **más lenta**, más el retardo de los registros de segmentación:

$$t_{ciclo} = \text{máx}(t_{IF}, t_{ID}, t_{EX}, t_{MEM}, t_{WB}) + t_{registro}$$

Si las etapas no están equilibradas, las más rápidas desaprovechan parte del ciclo.

5.2. Tiempo de ejecución y aceleración

Para n instrucciones en un pipeline de k etapas, sin riesgos:

$$T_{\text{sin segmentar}} = n \cdot T_{\text{instrucción}} \qquad T_{\text{segmentado}} = (k + n - 1) \cdot t_{ciclo}$$

Los k − 1 ciclos adicionales corresponden al llenado. Si las etapas fueran exactamente iguales ($T_{\text{instrucción}} = k \cdot t_{ciclo}$) y los registros no añadieran retardo, la aceleración sería:

$$S = T_{sin} / T_{seg} = n \cdot k / (k + n - 1) \;\rightarrow\; k \quad \text{cuando } n \rightarrow \infty$$

Es decir, el límite teórico de la aceleración es el número de etapas. Por ejemplo, con k = 5 y n = 100: S = 500 / 104 ≈ 4,81.

5.3. CPI real y tiempo de CPU

Cuando hay riesgos, el pipeline pierde ciclos (detenciones o *stalls*):

$$CPI_{real} = CPI_{ideal} + \text{ciclos de detención por instrucción} = 1 + \Sigma\ (\text{frecuencia del evento} \times \text{ciclos perdidos})$$

$$T_{CPU} = N_{instrucciones} \cdot CPI_{real} \cdot t_{ciclo}$$

Tres factores impiden alcanzar la aceleración ideal: las etapas desequilibradas, el retardo de los registros de segmentación y los riesgos. El ejercicio 1 aplica estas fórmulas.

6. Riesgos (hazards)

Un riesgo es una situación que impide que la siguiente instrucción se ejecute en el ciclo que le corresponde. Hay tres tipos:

Tipo	Causa	Ejemplo en RISC-V	Solución principal
Estructural	Dos etapas necesitan el mismo recurso en el mismo ciclo	`lw` en MEM y otra instrucción en IF con una única memoria	Duplicar recursos (cachés de instrucciones y datos separadas)
De datos	Una instrucción necesita un dato que otra anterior todavía no ha producido	`add x5, …` seguida de `sub x8, x5, …`	Anticipación, detenciones, reordenación
De control	No se sabe qué instrucción sigue hasta resolver un salto	`beq x5, x0, L`	Predicción de saltos, resolución temprana

6.1. Riesgos estructurales

Ocurren cuando dos o más etapas del pipeline compiten por el mismo recurso hardware en el mismo ciclo.

Ejemplo típico: una única memoria para instrucciones y datos (estilo von Neumann) con un solo puerto. En el ciclo 4, `lw` está en MEM (lee un dato) y la cuarta instrucción debería entrar en IF (leer una instrucción). Como no pueden atenderse los dos accesos a la vez, la cuarta instrucción espera un ciclo.

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9
`lw x5, 0(x10)`	IF	ID	EX	MEM	WB				
`add x6, x7, x8`		IF	ID	EX	MEM	WB			
`sub x9, x7, x8`			IF	ID	EX	MEM	WB		
`or x11, x7, x8`				—	IF	ID	EX	MEM	WB

acceso a memoria en conflicto ciclo perdido

Diagrama 2. Con una única memoria, el acceso a datos de lw (ciclo 4) impide buscar la cuarta instrucción, que se retrasa un ciclo.

Soluciones habituales:

- **Duplicar recursos**: memoria (caché) de instrucciones separada de la de datos —es lo que hace la ruta de datos de referencia de RISC-V—, sumadores propios para PC + 4 y para el destino de los saltos, y un banco de registros con dos puertos de lectura y uno de escritura.
- **Repartir el uso en el tiempo**: el banco de registros escribe en la primera mitad del ciclo y lee en la segunda.
- **Detener el pipeline** cuando el conflicto es inevitable (solución de último recurso).

En el pipeline clásico de RISC-V, con cachés separadas, no hay riesgos estructurales, por lo que no se consideran en el resto del documento.

6.2. Riesgos de datos

Aparecen cuando **una instrucción necesita un dato que todavía no ha sido producido o escrito por una instrucción anterior**.

6.2.1. Tipos de dependencias

Tipo	Significado	Ejemplo RISC-V	¿Riesgo en el pipeline de 5 etapas?
RAW (*Read After Write*) – dependencia verdadera	I2 lee un registro que escribe I1	`add x5, x6, x7` → `sub x8, x5, x9`	**Sí.** Es el único tipo que provoca riesgos en este pipeline.
WAR (*Write After Read*) – antidependencia	I2 escribe un registro que lee I1	`add x8, x5, x9` → `addi x5, x0, 1`	No: las lecturas (ID) de una instrucción siempre ocurren antes que las escrituras (WB) de las posteriores.
WAW (*Write After Write*) – dependencia de salida	I1 e I2 escriben el mismo registro	`lw x5, 0(x10)` → `addi x5, x0, 1`	No: todas las escrituras se hacen en WB y en orden.

WAR y WAW sólo se convierten en riesgos en procesadores que ejecutan fuera de orden o con etapas de distinta duración; allí se resuelven con renombrado de registros (apartado 8).

6.2.2. Ejemplo de riesgo RAW

```
add  x5, x6, x7      # x5 = x6 + x7
sub  x8, x5, x9      # necesita el valor NUEVO de x5
```

`add` escribe x5 en WB (ciclo 5), pero `sub` lo leería en ID en el ciclo 3. Si no se hace nada, `sub` usaría el valor antiguo de x5 y el resultado sería incorrecto.

6.2.3. Técnicas para resolver los riesgos de datos

a) Detención del pipeline (stalling, burbujas)

- La unidad de detección de riesgos retiene la instrucción dependiente en ID hasta que el dato está disponible; las instrucciones posteriores también se quedan quietas.
- Se implementa no escribiendo el PC ni el registro IF/ID durante ese ciclo e insertando una **burbuja** en ID/EX: una instrucción con todas las señales de control a 0, equivalente a `nop` (`addi x0, x0, 0`).
- Es fácil de implementar, pero reduce el rendimiento porque se pierden ciclos.

Instrucción \ ciclo	1	2	3	4	5	6	7	8
`add x5, x6, x7`	IF	ID	EX	MEM	WB			
`sub x8, x5, x9`		IF	ID	*ID*	*ID*	EX	MEM	WB

detención (la instrucción repite etapa)

Diagrama 3. Sin anticipación, sub espera en ID hasta el ciclo 5, en el que add escribe x5 (primera mitad) y sub lo lee (segunda mitad): 2 ciclos de detención.

b) Anticipación (forwarding / bypassing)

- El resultado de `add` existe al final de su etapa EX (ciclo 3), aunque no se escriba en x5 hasta WB. En lugar de esperar, se toma del registro de segmentación donde se encuentra (EX/MEM o MEM/WB) y se **reenvía** a la entrada de la ALU de la instrucción que lo necesita.
- En el ejemplo anterior, el resultado de `add` se anticipa desde EX/MEM hasta la etapa EX de `sub` en el ciclo 4: no hace falta ninguna detención.

Instrucción \ ciclo	1	2	3	4	5	6	7	8
`add x5, x6, x7`	IF	ID	EX	MEM	WB			
`sub x8, x5, x9`		IF	ID	EX	MEM	WB		

Diagrama 4. Con anticipación EX/MEM → EX, sub no se detiene: 6 ciclos en lugar de 8.

- Requiere una **unidad de anticipación** y multiplexores en las entradas de la ALU (Figura 1). Las condiciones que comprueba son:

```
# Riesgo EX: la instrucción anterior produce el dato
si (EX/MEM.RegWrite y EX/MEM.rd ≠ 0 y EX/MEM.rd = ID/EX.rs1)
      → entrada A de la ALU ← EX/MEM.resultado
# Riesgo MEM: la instrucción de dos posiciones antes produce el dato
si (MEM/WB.RegWrite y MEM/WB.rd ≠ 0 y MEM/WB.rd = ID/EX.rs1
    y no hay riesgo EX sobre rs1)
      → entrada A de la ALU ← MEM/WB.resultado
# Las mismas condiciones con rs2 controlan la entrada B
```

- La comprobación rd ≠ 0 evita anticipar escrituras en x0. Si se cumplen las dos condiciones a la vez, tiene prioridad EX/MEM porque contiene el valor más reciente.
- Reduce mucho las detenciones, a cambio de lógica adicional de detección y de los multiplexores de selección.

c) Riesgo de carga-uso (load-use)

La anticipación no lo resuelve todo. Una carga no tiene el dato hasta el final de MEM; si la instrucción siguiente lo necesita al principio de EX, el dato «llegaría tarde». Hace falta **una detención de un ciclo** y, después, anticipar desde MEM/WB.

```
lw   x5, 0(x10)
add  x6, x5, x7      # usa x5 justo después de cargarlo
```

Instrucción \ ciclo	1	2	3	4	5	6	7	8
`lw x5, 0(x10)`	IF	ID	EX	MEM	WB			
`add x6, x5, x7`		IF	ID	*ID*	EX	MEM	WB	

detención (la instrucción repite etapa)

Diagrama 5. Riesgo de carga-uso: 1 ciclo de detención incluso con anticipación (MEM/WB → EX en el ciclo 6).

La unidad de detección de riesgos, en ID, comprueba:

```
si (ID/EX.MemRead y (ID/EX.rd = IF/ID.rs1 o ID/EX.rd = IF/ID.rs2))
      → detener 1 ciclo (burbuja en ID/EX; PC e IF/ID no se actualizan)
```

d) Reordenación de instrucciones (por el compilador o el programador)

- Consiste en colocar instrucciones independientes entre dos instrucciones dependientes para «dar tiempo» a que el dato esté listo, sobre todo detrás de las cargas.
- No siempre es posible (puede que no haya instrucciones independientes que mover), pero cuando lo es, mejora el aprovechamiento del pipeline sin cambiar el hardware. Se ve en detalle en el ejercicio 4.

6.2.4. Resumen: ciclos de detención por cada dependencia RAW

Según la distancia entre la instrucción productora y la consumidora (distancia 1 = instrucciones consecutivas), en el pipeline de 5 etapas con escritura y lectura del banco de registros en el mismo ciclo:

Instrucción productora	Anticipación	Distancia 1	Distancia 2	Distancia ≥ 3
Aritmético-lógica	No	2	1	0
Aritmético-lógica	Sí	0	0	0
Carga (`lw`)	No	2	1	0
Carga (`lw`)	Sí	**1**	0	0

6.3. Riesgos de control

Se producen por las **instrucciones de salto**: el pipeline no sabe qué instrucción sigue hasta que el salto se resuelve (si se toma o no y a qué dirección va). Mientras tanto, IF sigue buscando instrucciones.

Si el pipeline ya ha empezado a ejecutar las instrucciones que siguen secuencialmente al salto y resulta que el salto se toma, esas instrucciones no deben completarse: hay que **anularlas** (*flush*), convirtiéndolas en `nop`, y buscar la instrucción de destino. Los ciclos que ocupaban se pierden.

En RISC-V las instrucciones de control son:

- **Saltos condicionales** (formato B): `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`. Destino = PC + inmediato (±4 KiB).
- **Salto incondicional** `jal rd, etiqueta` (formato J): destino = PC + inmediato (±1 MiB); guarda PC + 4 en rd. El destino no depende de ningún registro.
- **Salto indirecto** `jalr rd, imm(rs1)` (formato I): destino = rs1 + imm, con el bit 0 puesto a cero. Se usa para retornar de funciones (`ret` equivale a `jalr x0, 0(x1)`) y para llamar a través de punteros a función.

6.3.1. Penalización según la etapa en que se resuelve el salto

Etapa en la que se resuelve el salto	Instrucciones buscadas por el camino equivocado	Ciclos perdidos si el salto se toma
MEM (primer diseño de Patterson y Hennessy)	3	3
EX (diseño de referencia de este documento)	**2**	**2**
ID (comparador adelantado)	1	1

Ejemplo: salto tomado con predicción «no tomado» y resolución en EX.

```
0x100        addi x5, x0, 0      # x5 = 0
0x104        beq  x5, x0, L1     # x5 = 0 → el salto SE TOMA
0x108        addi x6, x0, 1      # camino secuencial (incorrecto)
0x10C        addi x7, x0, 2      # camino secuencial (incorrecto)
             ...
0x120  L1:   addi x8, x0, 3      # destino del salto
```

Instrucción \ ciclo	1	2	3	4	5	6	7	8	9
`addi x5, x0, 0`	IF	ID	EX	MEM	WB				
`beq  x5, x0, L1`		IF	ID	EX	MEM	WB			
`addi x6, x0, 1`			~~IF~~	~~ID~~					
`addi x7, x0, 2`				~~IF~~					
`L1: addi x8, x0, 3`					IF	ID	EX	MEM	WB

instrucción anulada (flush)

Diagrama 6. El salto se resuelve en EX (ciclo 4). Las dos instrucciones del camino secuencial se anulan y L1 se busca en el ciclo 5: se pierden 2 ciclos. El valor de x5 llega a beq por anticipación EX/MEM → EX.

6.3.2. Técnicas para reducir los riesgos de control

a) Detener el pipeline hasta resolver el salto

- Solución sencilla: tras cada salto se detiene la búsqueda de instrucciones hasta saber si se toma o no.
- Funciona, pero se paga la penalización completa en **todos** los saltos, incluso en los que no se toman. Resulta muy costoso cuando hay muchos saltos, por ejemplo en bucles.

b) Resolver el salto antes (en ID)

- Se añaden en ID un comparador de igualdad y el sumador del destino, de modo que la penalización baja a 1 ciclo.
- A cambio, el salto necesita sus operandos ya en ID: si depende de la instrucción inmediatamente anterior hay que detenerlo 1 ciclo (2 si depende de una carga), y hace falta anticipación hacia ID. El ejercicio 5 lo muestra.
- `jal` es un caso favorable: su destino sólo depende del PC y del inmediato, así que puede calcularse en ID (o predecirse ya en IF).

c) Ranuras de retardo (delay slots)

- Estrategia clásica de algunas arquitecturas RISC antiguas (MIPS original, SPARC).
- Se define que la instrucción inmediatamente posterior al salto **siempre se ejecuta**, se tome o no el salto. El compilador intenta colocar ahí una instrucción útil; si no la encuentra, pone un nop.
- En pipelines profundos o superescalares la penalización es de muchos ciclos y una sola ranura apenas ayuda, pero la ISA queda atada a ella para siempre. Por eso **RISC-V no tiene delay slots**.

d) Predicción de saltos (branch prediction)

Consiste en adivinar el resultado de un salto antes de conocerlo con certeza, para que el pipeline siga buscando instrucciones en lugar de detenerse. Hay que predecir dos cosas:

- ¿Se tomará el salto (*taken*) o no (*not taken*)?
- Si se toma, ¿cuál es la dirección de destino?

El procesador sigue llenando el pipeline según la predicción. Si acierta, no pierde ningún ciclo; si falla, anula las instrucciones buscadas por error (*flush*), busca las correctas y paga la penalización.

Predicción estática (fija; no depende de lo que ocurra durante la ejecución):

- **Predecir siempre «no tomado»**: se sigue buscando secuencialmente, como en el diagrama 6. Su coste hardware es prácticamente nulo.
- **Hacia atrás tomado, hacia delante no tomado** (BTFN): los saltos hacia atrás suelen cerrar bucles y casi siempre se toman. La especificación de RISC-V recomienda al software asumir este comportamiento, al menos la primera vez que se ejecuta cada salto, y organizar el código para que el camino más frecuente sea el secuencial.

Predicción dinámica (aprende del comportamiento pasado de cada salto):

- **Tabla de historia de saltos (BHT) de 1 bit**: recuerda si el salto se tomó la última vez. En un bucle falla dos veces por cada ejecución completa: en la última iteración y en la primera de la siguiente vez.
- **Contador saturado de 2 bits**: hacen falta dos fallos seguidos para cambiar la predicción, así que en un bucle sólo falla a la salida (Figura 2).
- **Buffer de destinos de salto (BTB)**: pequeña caché indexada por el PC que guarda la dirección destino de los saltos tomados. Permite redirigir la búsqueda ya en IF, sin perder ciclos si la predicción acierta.
- **Pila de direcciones de retorno (RAS)**: predice el destino de `jalr` cuando se usa como `ret`, apilando PC + 4 en cada llamada.
- Los procesadores actuales combinan varios predictores (de correlación global, de torneo, TAGE…) y superan el 95 % de aciertos en muchos programas.

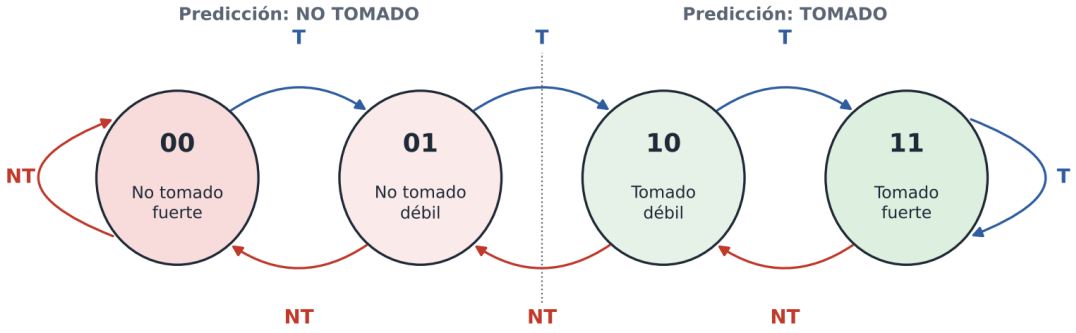


Figura 2. Predictor de 2 bits (contador saturado). T = salto tomado; NT = salto no tomado.

6.3.3. Coste de las predicciones fallidas

Cada salto mal predicho provoca:

- Vaciar parcialmente el pipeline (*flush*).
- Volver a buscar las instrucciones correctas.
- Perder varios ciclos: la penalización por fallo de predicción (*branch misprediction penalty*). En el pipeline de 5 etapas es de 1 a 3 ciclos según dónde se resuelva el salto; en procesadores con 15–20 etapas puede superar los 10 ciclos.

Su efecto sobre el CPI es:

$$\text{CPI} = 1 + f_{\text{saltos}} \times \text{tasa de fallo} \times \text{penalización}$$

Por ejemplo, si el 20 % de las instrucciones son saltos y la penalización es de 2 ciclos, detener el pipeline en todos los saltos añade 0,20 × 2 = 0,40 ciclos por instrucción (CPI = 1,40), mientras que con un predictor que acierta el 90 % sólo se añaden 0,20 × 0,10 × 2 = 0,04 (CPI = 1,04).

Con una buena predicción:

- Se reduce el número de detenciones y burbujas debidas a los saltos.

- El CPI real se aproxima al CPI ideal (sin pérdidas por control).
- La mejora se nota especialmente en programas con muchos bucles y ramas condicionales.

7. Más allá del pipeline clásico

- **Pipelines más profundos** (*superpipelining*): más etapas y más cortas permiten un reloj más rápido, pero aumentan la penalización por fallo de predicción y el número de riesgos.
- **Procesadores superescalares**: lanzan varias instrucciones por ciclo, de modo que el CPI puede ser menor que 1 (se suele hablar de IPC, instrucciones por ciclo).
- **Ejecución fuera de orden**: las instrucciones se ejecutan en cuanto tienen sus operandos. Aquí sí aparecen los riesgos WAR y WAW, que se resuelven con **renombrado de registros** y un búfer de reordenación (ROB) que confirma los resultados en orden.
- En el ecosistema RISC-V hay ejemplos de ambos estilos: núcleos de 5 etapas en orden, como Rocket (Berkeley), y núcleos superescalares fuera de orden, como BOOM (Berkeley).

8. Ideas clave

- La segmentación mejora la productividad (idealmente, CPI = 1), no la latencia de cada instrucción.
- El tiempo de ciclo lo fija la etapa más lenta más el retardo de los registros de segmentación; la aceleración máxima teórica es el número de etapas.
- Riesgos **estructurales**: se eliminan duplicando recursos (en RISC-V, cachés de instrucciones y datos separadas).
- Riesgos de **datos**: en el pipeline de 5 etapas sólo importan los RAW. La anticipación resuelve casi todos; la carga-uso sigue costando 1 ciclo, que el compilador puede ocultar reordenando.
- Riesgos de **control**: la penalización depende de la etapa en que se resuelve el salto. La predicción (estática o dinámica, con BTB) evita la mayor parte de las pérdidas; RISC-V no usa delay slots.
- Fórmula de trabajo: CPI = 1 + Σ (frecuencia del evento × ciclos perdidos).

Bibliografía

- D. A. Patterson y J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, Morgan Kaufmann, capítulo 4.
- *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V International.